LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 6 – INHERITANCE



NAMA: HUMAIRAH NURHAMIDAH

NIM: 103132400038

KELAS: PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Inheritance (pewarisan) adalah salah satu pilar OOP yang memungkinkan sebuah kelas diturunkan dari kelas lain sehingga mewarisi atribut dan method-nya. Hubungannya disebut "is-a relationship", dan tujuannya menghindari duplikasi kode. Kelas yang menurunkan disebut superclass, sedangkan kelas yang diturunkan disebut subclass. Di Java, pewarisan ditulis dengan keyword extends. Atribut private tidak bisa diakses langsung oleh subclass, sehingga biasanya disediakan getter.

Keyword super dipakai untuk mengakses anggota superclass. super(...) memanggil constructor induk dan harus ada di baris pertama constructor, sedangkan super.namaMethod() memanggil versi method milik induk.

Overloading adalah pembuatan beberapa method bernama sama dalam satu kelas dengan parameter yang berbeda. Overriding adalah pendefinisian ulang method superclass di subclass dengan signature yang sama, sehingga versi subclass yang dijalankan. Polymorphism memungkinkan variabel bertipe superclass menampung objek subclass, dan method yang berjalan mengikuti objek aslinya.

B. Guided

1. guided01

A.java

```
package guided01;


public class A {
    int x;
    int y;


    void tampilXY(){
        System.out.println("Nilai x = "+x+", y = "+y);
    }
}
```

B.java

```
package guided01;

public class B extends A{
    int z;
    int hasil;

    void jumlahXYZ(){
        hasil = (x+y+z);

        System.out.println("Jumlah (x+y+z) = " + hasil);
    }
}
```

Main.java

```
package guided01;

public class Main {
    public static void main(String[] args){

        A superclass = new A();
        B subclass = new B();

        superclass.x = 3;
        superclass.y = 4;

        System.out.println( "Superclass: ");
        superclass.tampilXY();

        subclass.x =1;
        subclass.y = 2;
        subclass.z = 5;

        System.out.println( "Subclass: ");
        subclass.jumlahXYZ();
    }

}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Superclass:
Nilai x = 3, y = 4
Subclass:
Jumlah (x+y+z) = 8
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

2. guided02

Employee.java

```
package guided02;

public class Employee {
    private String nama;
    String departemen;

    public Employee(String nama) {
        this.nama = nama;
    }

    public void tampilNama() {
        System.out.println("Nama : " + nama);
    }
}
```

Manager.java

```
package guided02;

public class Manager extends Employee{

    private String alamat;

    public Manager(String nama, String alamat){
        super(nama);
        this.alamat = alamat;
    }

    public void tampilInformasi(){
        super.tampilNama();
        System.out.println("alamat: " + alamat);
        System.out.println("departemen: " + super.departemen);
    }
```

```
}
```

Main.java

```
package guided02;

public class Main {

    public static void main(String[] args){
        Manager humaira = new Manager("Humaira", "Tegal");
        humaira.departemen = "Kominfo";
        humaira.tampilInformasi();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Nama : Humaira
alamat: Tegal
departemen: Kominfo
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

3. Guided03

Lagu.java

```
package guided03;

public class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1, String param2) {
        judul = param1;
        pencipta = param2;
    }

    void tampilkan() {
```

```
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);

    }
}
```

Main.java

```
package guided03;

public class Main {
    public static void main(String[] args){
        Lagu d,e;
        d = new Lagu();
        e = new Lagu();

        d.IsiParam("Magnolia");
        e.IsiParam("33x", "Perunggu");

        d.tampilkan();
        e.tampilkan();
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Judul : Magnolia, pencipta : Tidak dikenal
Judul : 33x, pencipta : Perunggu
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

4. guided04

   A.java

```
package guided04;

public class A {

    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}
```

B.java

```
package guided04;

public class B extends A {

    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}
```

Main.java

```
package guided04;

public class Main {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output:

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Method milik class A dipanggil...
Method milik class B dipanggil...
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

C. Unguided

Sebuah tim *machine learning* ingin membuat program untuk membandingkan estimasi biaya komputasi dari beberapa jenis model yang akan dilatih. Ada tiga jenis model: model regresi, model klasifikasi, dan model klasifikasi berbasis *deep learning* yang merupakan pengembangan dari model klasifikasi biasa. Buatlah program Java dengan ketentuan penamaan kelas, atribut, dan method ditentukan sendiri, dan semua atribut bersifat private.

1. **Kelas induk (model umum)**
    - Menyimpan nama model (teks) dan jumlah data latih (bilangan bulat).
    - Memiliki constructor yang menerima kedua nilai tersebut, serta cara bagi kelas lain untuk membaca jumlah data latih.
    - Memiliki method untuk menampilkan informasi.
    - Memiliki dua method penghitung biaya komputasi dengan nama yang sama (overloading): tanpa parameter (biaya = jumlah data latih × 2), dan dengan satu parameter bilangan bulat berupa jumlah epoch (biaya = jumlah data latih × epoch).
2. **Kelas turunan pertama (model regresi)**
    - Mewarisi kelas induk dan menambahkan atribut jumlah fitur.
    - Constructor menerima nama, jumlah data latih, dan jumlah fitur, dengan memanggil constructor kelas induk.
    - Override method tampil informasi dari kelas induk.
    - Override method hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) + (jumlah fitur × 10).
    - Tambahkan satu method baru dengan nama yang sama dengan method penghitung biaya, yang menerima dua parameter bilangan bulat (jumlah epoch dan ukuran batch): biaya = jumlah data latih × epoch ÷ ukuran batch. Dengan demikian, objek kelas ini memiliki tiga versi method penghitung biaya.
3. **Kelas turunan kedua (model klasifikasi)**
    - Mewarisi kelas induk dan menambahkan atribut jumlah kelas.
    - Constructor menerima nama, jumlah data latih, dan jumlah kelas, dengan memanggil constructor kelas induk.
    - Override method tampil informasi dari kelas induk.
    - Override method hitung biaya tanpa parameter: biaya = (biaya versi kelas induk) × jumlah kelas.
    - Versi penghitung biaya dengan parameter epoch tidak di-override.
4. **Kelas turunan ketiga (model klasifikasi deep learning)**
    - Mewarisi kelas turunan kedua (bukan kelas induk langsung) dan menambahkan atribut jumlah layer.
    - Constructor menerima nama, jumlah data latih, jumlah kelas, dan jumlah layer, dengan memanggil constructor kelas di atasnya.
    - Override method tampil informasi dari kelas di atasnya.
    - Override method hitung biaya tanpa parameter: biaya = (biaya versi kelas di atasnya) + (jumlah layer × 200).
5. **Class Main**
    - Buat tiga objek: Regresi (“Regresi Linier”, 500 data latih, 4 fitur), Klasifikasi (“Decision Tree”, 800 data latih, 3 kelas), dan Klasifikasi *deep learning* (“Neural Network”, 1000 data latih, 5 kelas, 3 layer).

- Simpan ketiga objek dalam satu array bertipe kelas induk.
- Gunakan perulangan untuk setiap elemen array: tampilkan informasi model, tampilkan biaya komputasi, lalu jumlahkan seluruh biaya.
- Setelah perulangan, tampilkan total biaya.
- Tampilkan juga biaya 10 epoch untuk “Neural Network” (ambil dari array), biaya 10 epoch untuk “Regresi Linier”, dan biaya 10 epoch dengan ukuran batch 50 untuk “Regresi Linier”.

Jawaban :

ModelML

```
package Unguided;


public class ModelML {
    // Kelas induk
    private String nama;
    private int jumlahDataLatih;

    public ModelML(String nama, int jumlahDataLatih) {
        this.nama = nama;
        this.jumlahDataLatih = jumlahDataLatih;
    }

    public int getJumlahDataLatih() {
        return jumlahDataLatih;
    }

    public void tampilInfo() {
        System.out.println("Model: " + nama + " | Data latih: " + jumlahDataLatih);
    }
```

```
    // Overloading: tanpa parameter
    public int hitungBiaya() {
        return jumlahDataLatih * 2;
    }

    // Overloading: satu parameter (epoch)
    public int hitungBiaya(int epoch) {
        return jumlahDataLatih * epoch;
    }
}
```

ModelRegresi

```
package Unguided;

public class ModelRegresi extends ModelML {
    // Turunan pertama: model regresi
    private int jumlahFitur;

    public ModelRegresi(String nama, int jumlahDataLatih, int jumlahFitur) {
        super(nama, jumlahDataLatih);
        this.jumlahFitur = jumlahFitur;
    }

    @Override
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah fitur: " + jumlahFitur);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahFitur * 10);
    }

    // Overloading baru: dua parameter (epoch dan batch)
    public int hitungBiaya(int epoch, int batch) {
```

```
        return getJumlahDataLatih() * epoch / batch;
    }
}
```

ModelKlasifikasi

```
package Unguided;

public class ModelKlasifikasi extends ModelML {
    private int jumlahKelas;

    public ModelKlasifikasi(String nama, int jumlahDataLatih, int jumlahKelas) {
        super(nama, jumlahDataLatih);
        this.jumlahKelas = jumlahKelas;
    }

    @Override
    public void tampilInfo() {
        super.tampilInfo();
        System.out.println("Jumlah kelas: " + jumlahKelas);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() * jumlahKelas;
    }
    // hitungBiaya(int epoch) tidak di-override
}
```

ModelDeepLearning

```
package Unguided;

// Turunan ketiga: klasifikasi deep learning (mewarisi ModelKlasifikasi)
public class ModelDeepLearning extends ModelKlasifikasi {
    private int jumlahLayer;

    public ModelDeepLearning(String nama, int jumlahDataLatih, int jumlahKelas,
int jumlahLayer) {
        super(nama, jumlahDataLatih, jumlahKelas);
        this.jumlahLayer = jumlahLayer;
    }

    @Override
    public void tampilInfo() {
```

```
        super.tampilInfo();
        System.out.println("Jumlah layer: " + jumlahLayer);
    }

    @Override
    public int hitungBiaya() {
        return super.hitungBiaya() + (jumlahLayer * 200);
    }
}
```

Main

```
package Unguided;

public class Main {
    public static void main(String[] args) {
        ModelRegresi regresi = new ModelRegresi("Regresi Linier", 500, 4);

        ModelML[] daftarModel = {
            regresi,
            new ModelKlasifikasi("Decision Tree", 800, 3),
            new ModelDeepLearning("Neural Network", 1000, 5, 3)
        };

        int total = 0;
        for (ModelML m : daftarModel) {
            m.tampilInfo();
            int biaya = m.hitungBiaya();
            System.out.println("Biaya komputasi: " + biaya);
            System.out.println("---");
            total += biaya;
        }

        System.out.println("Total biaya: " + total);
        System.out.println("Biaya 10 epoch (Neural Network): " +
daftarModel[2].hitungBiaya(10));
        System.out.println("Biaya 10 epoch (Regresi Linier): " +
regresi.hitungBiaya(10));
        System.out.println("Biaya 10 epoch batch 50 (Regresi Linier): " +
regresi.hitungBiaya(10, 50));
    }
}
```

Output :

```
--- exec:3.6.3:exec (default-cli) @ ProjectModul06 ---
Model: Regresi Linier | Data latih: 500
Jumlah fitur: 4
Biaya komputasi: 1040
---
Model: Decision Tree | Data latih: 800
Jumlah kelas: 3
Biaya komputasi: 4800
---
Model: Neural Network | Data latih: 1000
Jumlah kelas: 5
Jumlah layer: 3
Biaya komputasi: 10600
---
Total biaya: 16440
Biaya 10 epoch (Neural Network): 10000
Biaya 10 epoch (Regresi Linier): 5000
Biaya 10 epoch batch 50 (Regresi Linier): 100
------------------------------------------------------------------------
BUILD SUCCESS
------------------------------------------------------------------------
```

Penjelasan :

Program ini terdiri dari lima kelas yang menghitung estimasi biaya komputasi tiga jenis model *machine learning*. Kelas induk ModelML menyimpan nama model dan jumlah data latih sebagai atribut private, dengan getter getJumlahDataLatih() agar kelas turunan tetap bisa membaca jumlah data latih. Kelas ini juga punya method tampilInfo() dan dua versi hitungBiaya() (overloading): tanpa parameter (data latih × 2) dan dengan parameter epoch (data latih × epoch).

ModelRegresi mewarisi ModelML dan menambah atribut jumlah fitur. Constructor-nya memanggil super(...) untuk mengisi atribut induk. Method tampilInfo() dan hitungBiaya() di-override dengan memanggil versi induk lalu menambahkan bagiannya sendiri (biaya induk + fitur × 10). Kelas ini juga menambah versi hitungBiaya(epoch, batch) sehingga totalnya ada tiga versi. ModelKlasifikasi bekerja dengan pola yang sama, dengan atribut jumlah kelas dan biaya induk dikali jumlah kelas. Versi epoch-nya tidak di-override, jadi memakai milik induk.

ModelDeepLearning mewarisi ModelKlasifikasi, sehingga membentuk pewarisan bertingkat. Ia menambah atribut jumlah layer, dan biayanya adalah biaya ModelKlasifikasi ditambah layer × 200. Pada Main, ketiga objek disimpan dalam array bertipe ModelML[] (polymorphism), sehingga di dalam perulangan setiap objek otomatis menjalankan method versinya sendiri. Biaya tiap model dijumlahkan menjadi total 16440 (1040 + 4800 + 10600). Objek regresi disimpan dalam variabel bertipe ModelRegresi supaya versi dua parameter (epoch dan batch) bisa dipanggil, menghasilkan 100. Dua pemanggilan 10 epoch lainnya memakai method warisan, hasilnya 10000 untuk Neural Network dan 5000 untuk Regresi Linier.

D. Kesimpulan

Dari Modul 6 dapat disimpulkan bahwa inheritance memungkinkan sebuah kelas mewarisi atribut dan method dari kelas lain melalui keyword extends, sehingga kode tidak perlu ditulis berulang dan hierarki kelas dari yang umum ke yang spesifik menjadi jelas. Keyword super menjadi penghubung antara subclass dan superclass, baik untuk memanggil constructor maupun method induk. Overloading dan overriding sama-sama memungkinkan satu nama method memiliki banyak perilaku, tetapi berbeda cara kerjanya. Overloading dibedakan oleh parameter dalam satu kelas, sedangkan overriding mengganti perilaku method warisan di subclass. Polymorphism melengkapi semuanya dengan memungkinkan variabel bertipe superclass menyimpan objek subclass, dengan method yang dijalankan mengikuti objek aslinya.

Pada soal unguided, semua konsep tersebut diterapkan dalam satu program estimasi biaya komputasi model *machine learning*. Pewarisan bertingkat (ModelML → ModelKlasifikasi → ModelDeepLearning) menunjukkan bahwa super bisa saling berantai sehingga perhitungan biaya terakumulasi di tiap tingkat. Overloading tampak pada tiga versi hitungBiaya, sedangkan overriding tampak pada tampilInfo() dan hitungBiaya() di setiap kelas turunan. Polymorphism terlihat pada array ModelML[] yang menampung tiga jenis model dan diproses dalam satu perulangan. Penggunaan atribut private dengan getter juga menjaga prinsip enkapsulasi, sementara kelas turunan tetap dapat bekerja dengan data induknya. Hasil akhirnya adalah total biaya 16440, dengan biaya 10 epoch masing-masing 10000 untuk Neural Network, 5000 untuk Regresi Linier, dan 100 untuk Regresi Linier dengan batch 50.